

Building a Universal Context Layer for Platform Users

Ignite with Delhi | Ignite Room

Background

Most platforms today store user data in silos: a signup form here, an activity log there. None of these fragments alone tell you who a user actually is on that platform: what they do, what they are good at, and how they behave over time. Meanwhile, a company or organizer often needs to answer a simple question, "Is this person in our user base, and what do we know about them?", without manually digging through dashboards, tables, or spreadsheets.

Problem Statement

Design and build a **Context Layer**: a system sitting above a platform's raw database that continuously builds a rich, structured understanding of each user, based on their activity on that specific platform (and, where relevant, external professional data). This context should be queryable through a conversational AI agent that lets a company or organizer ask open-ended questions about any user in their database and get a synthesized answer, not just raw fields pulled from a table.

Teams must pick **one platform use case of their choice** (for example, a hackathon listing platform, a food delivery app, a coding practice platform, a freelance marketplace, or a community app) and design the context layer specifically for that use case. What counts as meaningful "context" is entirely dependent on the use case chosen; there is no fixed schema.

What Changes By Use Case

- **Tech-oriented platforms** (a hackathon platform, a developer community, a coding practice platform): external professional context, such as LinkedIn (education, work history) and GitHub (repositories, languages, contribution activity), is relevant and should be pulled in alongside platform activity.
- **Non-tech platforms** (a food delivery app, for example): external social or professional scraping is not relevant and should be skipped. Context here comes purely from platform-native behavioral data, such as what cuisines a user orders, order frequency, spending patterns, and ratings given.

Teams should clearly state, upfront, which use case they are solving for and justify what sources of context make sense for it. This decision is itself part of the deliverable, not just an implementation detail.

Example Use Case

ILLUSTRATIVE EXAMPLE: HACKATHON LISTING PLATFORM

A hackathon listing platform wants a context layer over its users. For this use case:

- **Pre-built / external context:** LinkedIn (education, past roles) and GitHub (repositories, tech stack, contribution history).
- **Platform-native context:** number of hackathons participated in, projects submitted, mentoring or feedback scores received, events attended, and comments or interactions on the platform.

An organizer can then ask the agent: "Is Shiv in our database? If so, tell me about him: his background, what he has built, and how active he has been in our hackathons."

The agent responds with a synthesized answer pulling from both external and platform-native context, not a raw data export.

This is one worked example. Teams are free to pick any other platform and define their own relevant context sources following the same logic.

Core Requirements

- **Choose and justify a platform use case**, and define what "context" means for it.
- **Ingest context relevant to that use case:** external sources (LinkedIn, GitHub) only where the use case calls for it, and/or platform-native activity data.
- **Build a context layer above the raw database:** not just a lookup into existing user tables, but a structured, synthesized profile per user built from the ingested signals.
- **Expose it via a conversational agent** that a company or organizer-type user can query, including checking whether a given person exists in the platform's user base, and answering open-ended questions about them.
- **Keep it generalizable in design**, even though the demo targets one chosen use case.

Bonus Points

- Building a **matchmaking algorithm** on top of the context layer (for example, matching users with each other, or with opportunities, teams, or mentors, based on their context).
- Demonstrating **any other downstream application** built using the context layer, beyond just constructing it, showing that the layer is genuinely useful and not just a data pipeline.

Evaluation Criteria

Criteria	What we are looking for
Use-case clarity	Is the chosen platform use case and its context sources well justified?
Context quality	How well the synthesized context actually reflects the user, not just a data dump.
Agent quality	Accuracy and usefulness of answers to open-ended questions, including existence checks.
Architecture	Is the context layer cleanly built above the database, and could it generalize to other use cases?
Bonus execution	Quality of any matchmaking algorithm or downstream use-case demo.